

L06911 · 鸿蒙操作系统 (2026 秋)

第 3 章智能终端操作系统内核

内核形态·鸿蒙通用微内核·双内核架构

江西科技师范大学

万博洋

2026 年 9 月 16 日 (第 3 教学周)

版权与来源说明

使用声明

本课件仅供教学使用，以《智能终端操作系统与 OpenHarmony 实践》（陈海波等，电子工业出版社）为基准编写。

- 教材插图：按图号标注，来源为上述教材
- 网图：ARM 指南、OpenHarmony 文档、华为《HarmonyOS 使用指南》，均在图注标注来源
- 课程自制图标注“课程自制”；代码示例为示意代码，非鸿蒙源码
- 学术数据注明出处；本课件仅供教学使用

本讲目录

- 3.0 应用如何进入内核：中断、异常与系统调用
- 3.1 操作系统内核及其挑战
- 3.2 面向智能终端的鸿蒙通用微内核
- 3.3 终端双内核架构：富功能内核和安全内核
- 3.4 端侧 AI 对内核的新要求

其中 3.2 含论文解读：OSDI'24 生产级微内核 (Microkernel Goes General)。

本讲导学

导学

本讲围绕“内核该怎么设计”展开：先弄清应用如何进入内核（中断/异常/系统调用），再对比宏内核、微内核、混合内核，然后介绍鸿蒙通用微内核与终端双内核架构。

- 关键词：信任边界·IPC（Inter-Process Communication，进程间通信）·轻量级隔离·TrustZone（ARM 硬件隔离技术）
- 带着问题听课：微内核“小”，凭什么还能撑起手机？

上节回顾

- 第 2 讲四层技术架构：应用层 → 框架层 → 系统服务层 → 内核层
- 内核层提供基础调度、内存、IPC 与安全能力，是唯一直接管理硬件资源的层次
- 本讲深入内核：宏内核、微内核、混合内核，终端该怎么选？

第 1 部分

3.0 应用如何进入内核：中断、异常与系统调用

3.0 定义：应用如何进入内核

定义

应用平时运行在用户态，访问设备、申请资源或发生错误时，需要经过中断、异常、系统调用“陷入”内核，由内核处理后返回。

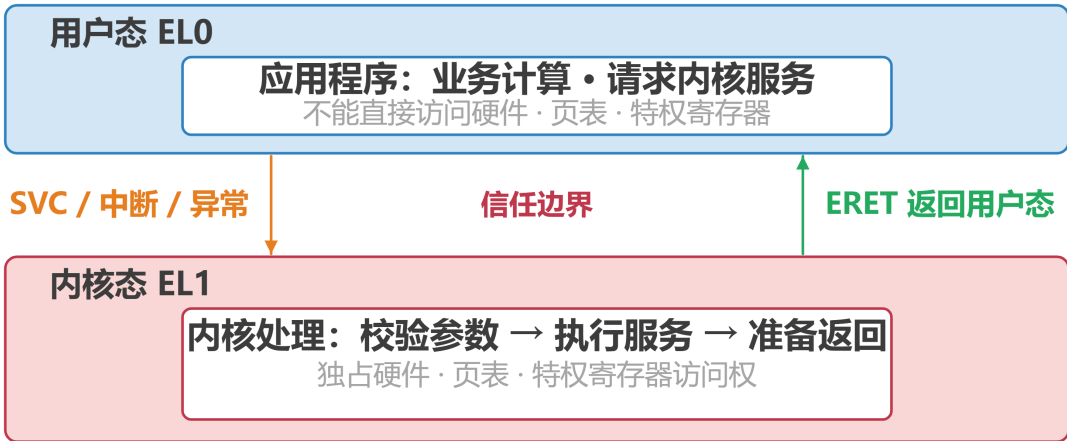
3.0 三条路径：中断、异常、系统调用

- 中断（异步）：外部设备随时触发，CPU 保存现场后进入中断处理，完成后返回
- 异常（同步）：执行中出错（除零、缺页、非法指令），由异常向量定位处理程序
- 系统调用：程序主动请求服务（read/write/fork），用户态 → 内核态 → 返回

3.0 特权级与信任边界

- AArch64: SVC 指令触发系统调用, EL0 (用户态) → EL1 (内核态); ERET 返回用户态
- 保存现场: 寄存器/PC/状态压栈, 处理完恢复, 保证程序继续执行
- 信任边界: 内核必须校验用户指针与参数, 防止恶意程序伪造地址

3.0 特权级切换与信任边界



3.0 SVC：进入内核的指令

SVC (Supervisor Call, 旧称 SWI)：用户态主动请求内核服务的标准入口——系统调用的硬件基础。执行 SVC #0 触发同步异常，CPU 硬件自动完成：

- ❶ 保存现场：处理器状态 PSTATE → SPSR_EL1
- ❷ 保存返回地址：下一条指令位置 → ELR_EL1
- ❸ 特权级切换：EL0 (用户) → EL1 (内核)，换用内核栈
- ❹ 跳到异常向量表的 SVC 入口；内核按调用号 (Linux 约定在 X8, 参数 X0-X5) 执行服务

3.0 ERET：返回用户态的指令

- **ERET** (Exception Return)：特权指令，用户态执行会触发异常——它是内核回用户态的“唯一正门”
- ① 恢复状态：SPSR_EL1 $\rightarrow$ PSTATE (回到 EL0、恢复中断屏蔽等)
- ② 恢复断点：ELR_EL1 $\rightarrow$ PC，从进入内核前的下一条指令继续
- 架构差异：AArch64 用 ERET；32 位 ARMv7 等效于 MOVS PC, LR

完整往返：一次 write 系统调用

准备参数 (X0-X5, 调用号 $\rightarrow$ X8) $\rightarrow$ SVC #0 进内核 $\rightarrow$ 内核执行 sys_write $\rightarrow$ ERET 恢复现场，回到用户态继续跑。

3.0 对比：三种进入方式

表: 中断 / 异常 / 系统调用对比

维度	中断	异常	系统调用
触发	外部异步	指令同步	程序主动
来源	设备/时钟	执行错误	用户代码
去向	中断向量	异常向量	固定编号
典型例子	定时器/网卡	缺页/除零	read/write

3.0 三条进入内核的路径



实验 3.0: /proc 进程现场

```
## 进程的内存档案 (谁维护? 内核)
PS E:\WelcomeHarmony> hdc shell head -14 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   27031
Ngid:   0
Pid:    27031

## 进程地址空间: 每行都是一次 mmap 的结果
PS E:\WelcomeHarmony> hdc shell head -6 /proc/self/maps
55d4f2129000-55d4f2152000 r--p 00000000 fe:00 612           /system/bin/toybox
55d4f2152000-55d4f2189000 r-xp 00028000 fe:00 612           /system/bin/toybox
55d4f2189000-55d4f218d000 r--p 0005e000 fe:00 612           /system/bin/toybox
55d4f218d000-55d4f2197000 rw-p 00061000 fe:00 612           /system/bin/toybox
55d4f2197000-55d4f219b000 rw-p 00000000 00:00 0           [anon:head.bss]
55d4f3bd6000-55d4f3bd7000 ---p 00000000 00:00 0           [heap]

## 内核线程与用户进程同框
PS E:\WelcomeHarmony> hdc shell ps -ef
UID          PID    PPID  C  STIME TTY          TIME CMD
root          1        0  1 12:31:00 ?        00:00:38 init --second-stage 4044158
root          2        0  0 12:31:00 ?        00:00:00 [kthreadd]
root          3        2  0 12:31:00 ?        00:00:00 [rcu_gp]
root          4        2  0 12:31:00 ?        00:00:00 [rcu_par_gp]
root          6        2  0 12:31:00 ?        00:00:00 [kworker/0:0H-events_highpri]
root          8        2  0 12:31:00 ?        00:00:00 [mm_percpu_wq]
root          9        2  0 12:31:00 ?        00:00:01 [ksoftirqd/0]
PS E:\OneDrive - jxstnu.edu.cn\江西科技师范大学 (课程教学材料) \2026-2027 (第一学期) \鸿蒙操作系统\教学slides\生成脚本ka
```

- ① /proc/self/status: 内核维护的进程档案 ② /proc/self/maps: 每行一次 mmap ③ 内核线程与用户进程同

框

第 2 部分

3.1 操作系统内核及其挑战

3.1 定义：内核及其挑战

定义

内核是操作系统中最核心、最靠近硬件的部分，负责 CPU 调度、内存管理、进程/线程管理、IPC 与设备访问等基本机制。其中 IPC (Inter-Process Communication, 进程间通信) 指进程之间传递数据、请求服务与同步执行的机制——微内核把大量功能移出内核后，服务之间的协作几乎全靠它。

3.1 三类内核形态

- 宏内核：内核包含全部核心功能，性能高、开发方便，但故障隔离弱、安全攻击面大
- 微内核：内核只保留最基本机制，服务移到用户态进程，隔离性强、安全面小
- 微内核代价：服务间通信频繁，IPC 性能成为关键挑战
- 混合内核：在宏内核与微内核之间取平衡，按场景混合设计
- 其他内核形态：外内核、多内核等研究探索

3.1 智能终端对内核的五大要求

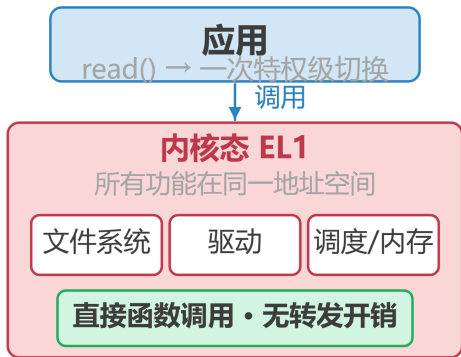
智能终端对内核的五大要求

- 高性能：调度 / IPC / 内存路径低时延，交互与游戏帧率才有保障
- 强隔离：应用间、应用与内核间故障不扩散，一个应用崩溃不能拖死系统
- 低功耗：空闲快速休眠、事件快速唤醒，续航是终端的生命线
- 高安全：从启动到运行建立完整信任链，保护指纹、密钥、支付
- 可扩展：同一内核覆盖手机 / 平板 / 车机 / 手表等多形态设备

来源：教材 3.1 “操作系统内核及其挑战”。

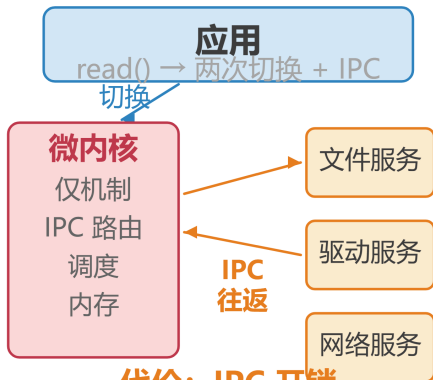
3.1 系统调用路径：宏内核 vs 微内核

宏内核：系统调用即函数调用



优点：快

微内核：系统调用经 IPC 转发



代价：IPC 开销

3.1 对比：三类内核形态

表: 三类内核形态对比

特性	宏内核	微内核	混合内核
功能范围	全部核心功能	最小机制	按需混合
性能	高	IPC 开销大	较高
隔离性	弱	强	中等
安全面	大	小	中等
典型系统	Linux、UNIX	seL4、鸿蒙微内核	Windows、macOS

3.1 三类内核形态对比

宏内核

文件系统

设备驱动

内存管理

进程管理

全部功能在内核态

微内核

最小化内核

调度 · IPC · 内存

用户态

文件系统（服务）

驱动（服务）

最小内核 + 服务上移

混合内核

内核态

核心 + 部分服务
(如图形、驱动)

其余服务在用户态

按场景混合取舍

性能：宏 > 混合 > 微 | 隔离与安全：微 > 混合 > 宏
典型系统：Linux / UNIX · seL4 / 鸿蒙微内核 · Windows / macOS

3.1 案例：装驱动与蓝屏

系统	内核形态	驱动安装	驱动出错后果
Linux	宏内核	插入.ko 内核模块	kernel panic, 整机重启
Windows	混合内核	加载.sys 驱动	蓝屏 BSOD, 整机重启
QNX / 鸿蒙	微内核	用户态驱动服务	重启该服务, 系统不倒

真实案例：2024 年 7 月 CrowdStrike 全球蓝屏

一个内核驱动文件缺陷，约 850 万台 Windows 蓝屏，航空、银行、医院瘫痪（微软估计）。根源：驱动在内核态；微内核移出驱动，换故障隔离。

第 3 部分

3.2 面向智能终端的鸿蒙通用微内核

3.2 定义：鸿蒙通用微内核

定义

鸿蒙通用微内核是 OpenHarmony 面向智能终端设计的微内核：内核尽可能小，只提供调度、内存、IPC、任务管理等必要机制，其余功能以服务形式运行在用户态。

3.2 设计目标与内核最小化

- 设计目标：通用（支持多种终端）、高效（低时延）、安全（强隔离）、可扩展
- 内核最小化：策略上移到用户态服务，内核只提供机制
- 高性能 IPC：为服务间高频调用设计快速路径，降低切换开销
- 轻量级隔离：按安全需求划分隔离等级（IC0/IC1/IC2 三级，见后文架构图）

IPC 快慢的四个决定因素

- ① **切换次数**：一次调用 = 用户态 → 内核 → 对端服务（模式切换 + 线程切换），同步往返再翻倍
- ② **数据拷贝**：消息经内核中转要拷两次，大消息代价线性涨；零拷贝（共享内存）才快
- ③ **调度与缓存**：唤醒对端线程要排队；跨核调用还要发核间中断、缓存变冷
- ④ **权限检查**：每次调用都要验权限；传统能力机制查找贵，鸿蒙“地址口令”压到最低

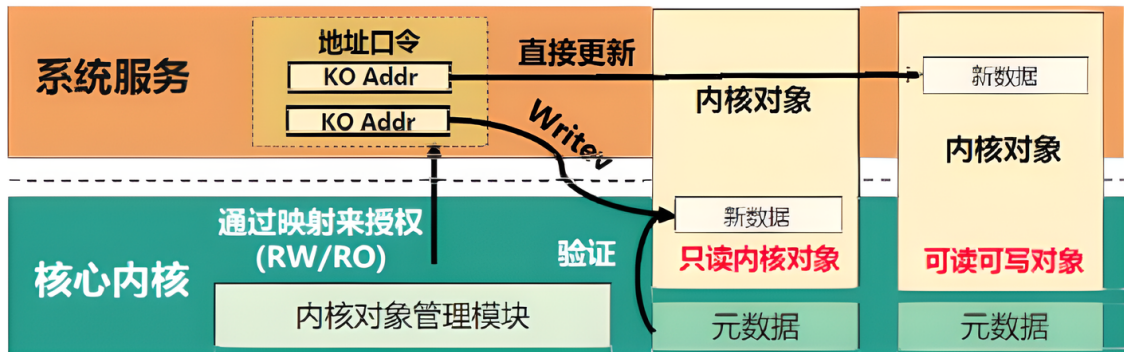
对照论文四个技术

合并服务（减切换）·无策略分页（消双记账）·地址口令（降检查）·驱动容器（免重写驱动）

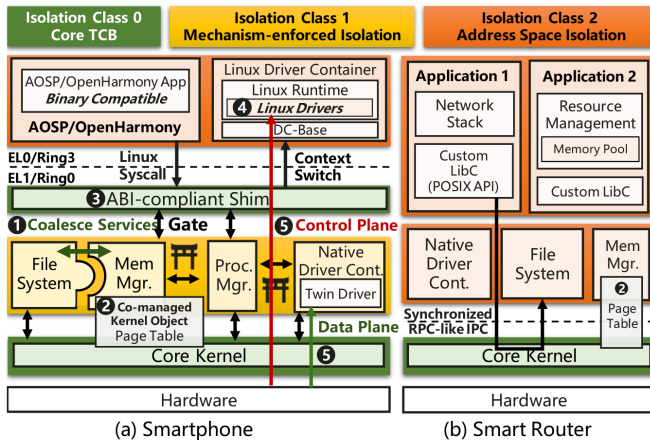
3.2 高性能 IPC 与轻量级隔离

- 地址口令 (Token) 机制：细粒度权限控制，替代传统能力列表的昂贵检查
- 鸿蒙内核架构：任务管理、内存管理、IPC、异常处理、系统调用等模块

3.2 地址口令机制（教材图 3-5）



3.2 鸿蒙微内核架构 (OSDI'24 论文图 3)



3.2 机制与服务分离

内核：只保留机制
任务调度 · 内存管理 · IPC
中断 · 安全 Token

IPC 调用
轻量隔离



服务：独立进程
文件系统 · 网络 · 驱动
多媒体 · 系统服务

机制与服务分离

极小化内核 → 更低时延 · 更强隔离 · 服务可复用

3.2 论文解读：走向通用

- 困境：微内核长于嵌入式与安全关键场景，走向手机、汽车等通用场景时遇到**性能与兼容性**两大挑战
- 鸿蒙微内核（HM）的选择：兼容 Linux API 与 ABI（应用二进制接口），直接复用 Linux 应用与驱动生态
- 重新审视微内核“传统智慧”：单次 IPC 快还不够——
 - IPC **频率**高：服务拆得越细，跨服务调用越频繁
 - 服务间**状态双记账**：内核与用户态服务各记一份，同步代价大
 - 能力（capability）机制**隐藏内核对象**，增加检查开销

来源：OSDI'24 论文（Chen 等，华为中央软件研究院 + 上海交大）；PDF 见 02-学术论文/。

3.2 论文解读：技术与验证

- 差异化隔离等级：按场景划分隔离强度 (IC0/IC1/IC2)，隔离也是要花性能的
- 灵活组合：服务可按部署场景组合，减少跨服务 IPC 次数
- 无策略内核分页：分页策略上移用户态，内核只保留机制
- 地址口令 (Address-Token) 访问控制：细粒度、低开销的权限检查
- 生产验证：已部署在**数千万台**设备——智能路由器、智能汽车、智能手机；可运行 AOSP 与 OpenHarmony 完整框架

引用：Haibo Chen et al. Microkernel Goes General: Performance and Compatibility in the HongMeng Production Microkernel. In Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), USENIX Association, 2024, pp. 464–485. (“数千万台”为论文原文 tens of millions of devices)

课堂思考：微内核把大部分功能移到用户态，为什么 IPC 性能会成为决定成败的关键？

先独立思考，再对照解答

结合本节讲过的机制与代价来推理。

解答：微内核把大部分功能移到用户态，为什么 IPC 性能会成为决定成败的关键？

参考解答

- 服务间调用都要经过 IPC，微内核下 IPC 成为系统调用主路径
- IPC 不快，微内核“小而安全”的优势就抵不过性能损失
- 鸿蒙微内核用快速路径、轻量抽象、高效资源管理与轻量级隔离压低时延

实验 3.2：系统服务真的在用户态

```
## 列出所有系统服务 (SA)
PS E:\WelcomeHarmony> hdc shell hidumper -ls
System ability list:
SystemAbilityManager      RenderService              AbilityManagerService
DataObserverMgr           AgentManagerService        AccountMgr
BundleMgr                 FormMgr                    ApplicationManagerService
AccessibilityManagerService UserIdmService              UserAuthService
AuthExecutorMgrService    PinAuthService              FaceAuthService
WifiDevice                WifiHotspot                 WifiP2p
WifiScan                  1125                        1126

## 服务内存占用：它们是普通进程
PS E:\WelcomeHarmony> hdc shell hidumper --mem
-----[memory]-----
Total Memory Usage by PID:
PID      Total Pss(xxx in SwapPss)  Total Vss  Total Rss  Total Uss  GL  Graph  Dma
PurgSum  PurgPin  Name
1        3440(0 in SwapPss) kB    37712 kB   7440 kB    3112 kB    0 kB  0 kB   0 kB
0 kB     0 kB    init
100      1349(0 in SwapPss) kB    12620 kB   4576 kB    1316 kB    0 kB  0 kB   0 kB
0 kB     0 kB    ueventd
108      951(0 in SwapPss) kB    24084 kB   4584 kB    912 kB     0 kB  0 kB   0 kB
0 kB     0 kB    faultloggerd
112      3698(0 in SwapPss) kB    25156 kB   8284 kB    3644 kB    0 kB  0 kB   0 kB
0 kB     0 kB    hilogd
123      3509(0 in SwapPss) kB    58816 kB   15296 kB    3200 kB    0 kB  0 kB   0 kB
0 kB     0 kB    storage_manager
PS E:\OneDrive - jxstnu.edu.cn\江西科技师范大学 (课程教学材料) \2026-2027 (第一学期) \鸿蒙操作系统\教学slides\生成脚本>
```

- ① hidumper -ls：几十个系统服务 (SA) 全部运行在用户态 ② hidumper --mem：它们是普通进程、有内存账单

第 4 部分

3.3 终端双内核架构：富功能内核和安全内核

3.3 定义：终端双内核架构

定义

智能终端采用“富功能内核 + 安全内核”双内核架构：富功能内核提供通用能力，安全内核基于 TrustZone 提供强隔离的可信执行环境。

3.3 TrustZone 双世界隔离

- 硬件基础：ARM TrustZone 将处理器划分为普通世界与安全世界
- 富功能内核运行在普通世界，承载完整 OS 功能与生态兼容
- 安全内核（OpenTrustee）运行在安全世界，保护密钥、指纹等敏感数据
- 基于 TrustZone 的实时内核保护：防篡改、防注入、保障系统完整性

3.3 双内核协作：安全监控器与价值

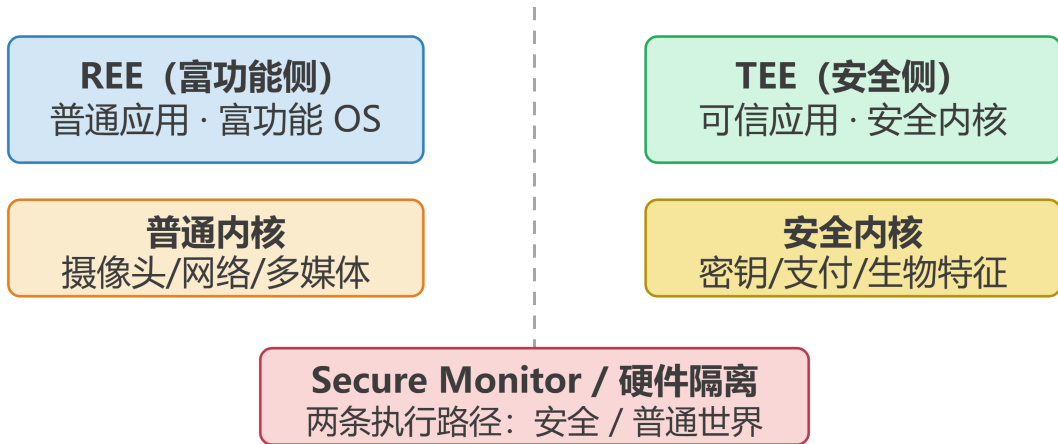
- 双内核协作：普通世界请求安全服务时，通过安全监控器切换
- 价值：把“安全”从软件策略提升到硬件隔离层面

3.3 对比：富功能内核与安全内核

表: 富功能内核与安全内核分工

维度	富功能内核（普通世界）	安全内核（安全世界）
职责	完整 OS 功能与生态兼容	密钥、指纹等敏感数据处理
隔离	与安全世界硬件隔离	TrustZone 强隔离
协作	请求安全服务	经安全监控器响应
价值	通用性与性能	安全性与可信执行

3.3 TrustZone 硬件隔离



实验 3.3：寻找安全世界

实验 3.3：寻找安全世界

tee 二进制存在（安全世界接口）

```
PS E:\WelcomeHarmony> hdc shell ls -l /system/bin/tee
ls: /system/bin/tee: Permission denied
```

尝试读取（被隔离，普通世界不可见）

```
PS E:\WelcomeHarmony> hdc shell cat /system/bin/tee
ELF, %d minute%s, C)$♦♦I♦♦L♦♦H♦♦1♦♦♦tH♦♦♦♦
```

安全相关服务进程

```
PS E:\WelcomeHarmony> hdc shell ps -ef
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	1	12:31:01	?	00:00:39	init --second-stage 4044158
root	2	0	0	12:31:01	?	00:00:00	[kthreadd]
root	3	2	0	12:31:01	?	00:00:00	[rcu_gp]
root	4	2	0	12:31:01	?	00:00:00	[rcu_par_gp]
root	6	2	0	12:31:01	?	00:00:00	[kworker/0:0H-events_highpri]

- ❶ tee 接口存在但 Permission denied ❷ ELF 头证明是接口程序 ❸ 普通世界 ps 看不到任何安全世界进程

第 5 部分

3.4 AI 时代的终端操作系统内核

3.4 定义：AI 时代的终端内核

定义

大模型上端侧对内核提出新要求：需要为 AI 任务提供差异化调度、内存与算力资源保障。

3.4 AI 时代的终端操作系统内核

- 端侧 AI 负载特征：推理任务密集、时延敏感、功耗预算有限
- 内核需要支持异构算力调度（CPU/GPU/NPU）
- 大模型内存占用大，需要高效的内存管理与按需加载
- 系统级智能助手与内核深度协同，感知场景动态调整策略
- 展望：AI 原生内核成为下一阶段竞争焦点

3.4 端侧 AI 负载特征与内核需求

表: 端侧 AI 负载特征 → 对内核的要求

负载特征	具体表现	内核需要提供
推理密集	大模型逐层计算, 算子并发度高	异构调度: CPU / GPU / NPU 协同
时延敏感	首 token 与逐 token 时延影响体感	优先级继承、实时任务抢占
内存大	模型常驻数百 MB 至数 GB	按需加载、内存压缩回收
功耗受限	端侧电池有限, 忌常驻高负载	能效优先调度、动态调频
数据敏感	语音 / 图像在端侧本地处理	内存隔离、可信执行环境

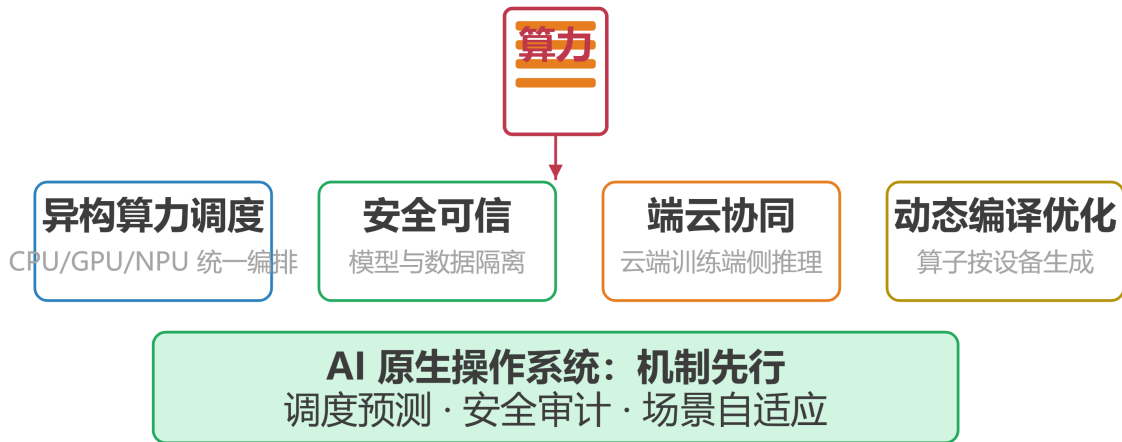
资料整理: 教材第 3 章与端侧 AI 推理实践。

3.4 异构算力调度：内核的新课题

- 传统内核只调度 CPU；AI 时代需统一调度 **CPU / GPU / NPU** 三类算力
- 目标从”公平与吞吐”扩展为”**时延可预期 + 能效最优**”
- 机制：任务亲和性绑定、算力域划分、算子级抢占、内存与算力联合分配
- 挑战：异构驱动差异大需统一抽象；模型演进快，调度策略需可配置
- 呼应前文：3.2 轻量级隔离保护模型与数据；3.3 可信执行环境承载敏感推理

来源：教材第 3 章；端侧 AI 部署实践（含 OpenHarmony 异构计算方向）。

3.4 AI 原生内核：机制先行



Take-away (本讲小结)

核心要点

- 3.0 三条进入内核的路径：中断（异步）、异常（同步）、系统调用（主动）
- 宏内核/微内核/混合内核的优缺点对比
- 鸿蒙通用微内核：小内核、高性能 IPC、轻量级隔离、地址口令
- OSDI'24：Linux 兼容 + 四大关键技术，数千万台设备生产验证
- 双内核架构：TrustZone + OpenTrustee；考点：三类内核对比各 3 点、IC 隔离等级

实验工具：hdc 命令行

- **hdc** (HarmonyOS Device Connector)：鸿蒙官方设备调试工具（类比 Android adb）——连接设备、执行 shell、装应用、传文件、抓日志
- 随 SDK 安装：<SDK>/openharmoney/toolchains/hdc.exe；不在 PATH 时用完整路径调用

三步连接模拟器

- ❶ `hdc list targets`：列出已连设备（模拟器未启动时显示 Empty）
- ❷ 列表为空则手动连接：`hdc tconn 127.0.0.1:5555`
- ❸ 再次 `list targets` 确认出现 `127.0.0.1:5555` 即连上

本讲所有实验均在 `hdc shell` 中执行；真机 USB 连接自动出现；实测版本 3.2.0d。

第 3 周实验卡：看见内核

本周必做 (30–60 分钟)

- 用 `hdc shell` 执行 `uname -a` 看内核版本、`ps -ef` 看用户态进程
- 用 `param get` 查询系统软件版本与 API 版本

AI 验证题 (课后 1 题)

- 让 AI 解释“中断和异常的区别”，用本讲三条判据（触发来源/同步异步/去向）核对 AI 的回答并截图

来源：课程《每周实验卡》；命令在 SDK 的 `toolchains` 目录下执行（实测示例见下页）。

第 3 周实验卡：实测命令与输出

```
PS E:\WelcomeHarmony> hdc list targets
127.0.0.1:5555

PS E:\WelcomeHarmony> hdc shell uname -a
Linux localhost 5.10.210 #1 SMP Tue Jul 21 15:31:32 CST 2026 x86_64 Toybox

PS E:\WelcomeHarmony> hdc shell ps -ef
UID          PID    PPID  C  STIME TTY          TIME CMD
root           1        0  1  12:31:00 ?        00:00:35 init --second-stage 4044158
root           2        0  0  12:31:00 ?        00:00:00 [kthreadd]
root           3        2  0  12:31:00 ?        00:00:00 [rcu_gp]
root           4        2  0  12:31:00 ?        00:00:00 [rcu_par_gp]
root           6        2  0  12:31:00 ?        00:00:00 [kworker/0:0H-events_highpri]
root           8        2  0  12:31:00 ?        00:00:00 [mm_percpu_wq]
root           9        2  0  12:31:00 ?        00:00:01 [ksoftirqd/0]
root          10        2  0  12:31:00 ?        00:00:18 [rcu_sched]

PS E:\WelcomeHarmony> hdc shell param get const.product.software.version
emulator 6.1.0.126(SP1DEV00E120R4P11)

PS E:\WelcomeHarmony> hdc shell param get const.ohos.apiversion
24

== Kernel 5.10.210 | API 24 | 193 processes ==
```

- ① 连接模拟器 ② `uname -a` 看内核版本 ③ `ps -ef` 看用户态进程 ④ `param get` 查询系统与 API 版本

实验 A：内核是系统的最后防线

实验 A：内核是系统的最后防线

内核自身信息：普通进程读不到

```
PS E:\WelcomeHarmony> hdc shell cat /proc/version
/bin/sh: cat: /proc/version: Permission denied
```

内核线程 vs 用户进程（方括号=内核线程）

```
PS E:\WelcomeHarmony> hdc shell ps -ef
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	1	12:31:00	?	00:00:35	init --second-stage 4044158
root	2	0	0	12:31:00	?	00:00:00	[kthreadd]
root	3	2	0	12:31:00	?	00:00:00	[rcu_gp]
root	4	2	0	12:31:00	?	00:00:00	[rcu_par_gp]
root	6	2	0	12:31:00	?	00:00:00	[kworker/0:0H-events_highpri]

内核线程不可 kill (PID 2 = kthreadd)

```
PS E:\WelcomeHarmony> hdc shell kill -9 2
/bin/sh: kill: 2: Operation not permitted
```

系统分区只读：内核拒绝写入

```
PS E:\WelcomeHarmony> hdc shell touch /system/hackme
touch: '/system/hackme': Read-only file system
```

== 以上操作全部被内核拒绝或受内核控制 ==

- ① 内核自身信息读不到 ② 方括号 = 内核线程 ③ 内核线程不可 kill ④ 系统分区只读

实验 B：内核安全机制

实验 B：内核安全机制

当前身份与 SELinux 安全上下文

```
PS E:\WelcomeHarmony> hdc shell id
uid=2000(shell) gid=2000(shell) groups=2000(shell),1006(file_manager),1007(log),1097(netsys_socket),3009(readproc) context=u:r:sh:s0
```

内核模块列表不可读

```
PS E:\WelcomeHarmony> hdc shell lsmod
lsmod: No file /proc/modules: Permission denied
```

内核参数不可篡改

```
PS E:\WelcomeHarmony> hdc shell param set const.product.model HACKED
Set parameter const.product.model HACKED fail! errMsg is:1001!
```

内核日志需特权

```
PS E:\WelcomeHarmony> hdc shell dmesg
dmesg: klogctl: Operation not permitted
```

== 以上操作全部被内核拒绝或受内核控制 ==

- ① SELinux 安全上下文 ② 内核模块不可读 ③ 内核参数不可篡改 ④ 内核日志需特权

实验 3.4：看见算力与调度

```
Tasks: 186 total,  2 running, 184 sleeping,  0 stopped,  0 zombie
      Mem:  4025344K total,  1933584K used,  2091760K free,    16024K buffers
      Swap:  2097148K total,   196608K used,  1900540K free,   661592K cached
400%cpu   0%user   0%nice   0%sys 400%idle   0%iow   0%irq   0%sirq   0%host
PS E:\OneDrive - jxstnu.edu.cn\江西科技师范大学（课程教学材料）\2026-2027（第一学期）\鸿蒙操作系统\教学slides\生成脚本>
```

❶ top: 188 个任务、4GB 内存、400%cpu (4 核) ❷ hidumper --mem: 内核为每个进程记账

能否控制或伤害内核?: 攻击面与防护

历史上的经典攻击面

- 内核符号表: 定位内核函数地址
- 物理内存设备 (/dev/mem): 直接改内核数据
- 内核参数 (sysctl): 篡改调度与安全策略
- 内核模块: 加载恶意代码进内核
- 系统调用接口: 畸形参数触发漏洞
- 设备驱动: 最多漏洞来源 (占内核漏洞多数)

对应的防护机制

- 权限位: 普通进程 CapEff=0 (零能力)
- 强制访问控制: SELinux 默认 Enforcing
- 只读分区: 系统与内核镜像不可写
- 内核最小化: 接口越少, 攻击面越小
- 地址口令: 细粒度校验每次访问
- 形式化验证: 微内核可证明安全性

下一个要点: 既然内核这么难攻, 攻击者会转向哪? ——转向用户态与驱动。

内核攻击面实测：全部被拒

```
PS E:\WelcomeHarmony> hdc shell head -2 /proc/kallsyms
head: /proc/kallsyms: Permission denied

## 写物理内存设备（经典提权路径）
PS E:\WelcomeHarmony> hdc shell ls -l /dev/mem
ls: /dev/mem: Permission denied

## 改内核参数（sysctl）
PS E:\WelcomeHarmony> hdc shell echo 99 > /proc/sys/kernel/panic
/bin/sh: can't create /proc/sys/kernel/panic: Permission denied

## 杀内核线程
PS E:\WelcomeHarmony> hdc shell kill -9 2
/bin/sh: kill: 2: Operation not permitted

## 加载内核模块
PS E:\WelcomeHarmony> hdc shell lsmod
lsmod: No file /proc/modules: Permission denied

## 看本进程的能力位
PS E:\WelcomeHarmony> hdc shell grep CapEff /proc/self/status
CapEff: 0000000000000000

## SELinux 强制模式
PS E:\WelcomeHarmony> hdc shell getenforce
Enforcing

== 结论：CapEff=0（零能力）+ SELinux Enforcing ==
== 普通进程没有任何直接控制内核的合法途径 ==
```

7 条路径全被拒；关键证据：CapEff=0（零能力）+ SELinux: Enforcing

延伸资源与实践

论文与文档

- **OSDI'24: Microkernel Goes General (鸿蒙生产级微内核):**
02-学术论
文/OSDI24-HongMeng-Microkernel-Goes-General.pdf
- **OSDI'24 演讲 Slides:** 02-学术论
文/OSDI24-HongMeng-Slides.pdf (架构总览)
- **OpenHarmony 内核子系统文档:**
<http://docs.openharmony.cn>

延伸资源与实践（续 1）

实验与预习

- 《计算机操作系统实践教程（OpenHarmony 内核平台）》：清华大学出版社，配套进程/调度/内存/文件/设备 9 个实验
- 课程网站：<https://wanboyang.github.io>（教学进度与课件直链）
- 预习：教材第 4 章“处理器调度”（下次课主题）

作业与预习

课后任务

- 完成三类内核对比表（隔离性、性能、复杂度、安全、典型系统）
- 提交第 3 周实验卡：hdc shell 观察截图 + AI 验证结论（下次课前）

下次课预告

预告

第 4 讲：内核机制（1）——CPU 调度与调频（DVFS/大小核）